



状态空间法

- 基于解答空间的问题表示和求解方法，即为**状态空间法**，它是以状态和算符为基础来表示和求解问题的。
- **状态**：问题在某一时刻所处的“位置”，“情况”等。根据问题所关心的因素，一般用向量形式表示，每一位表示一个因素。
- **算符（操作符）**：使状态发生改变的操作。
- 从某一**初始状态**开始，每次施加一个**操作符**，递进地建立**操作符序列**，直到达到**目标状态**为止。

状态空间法

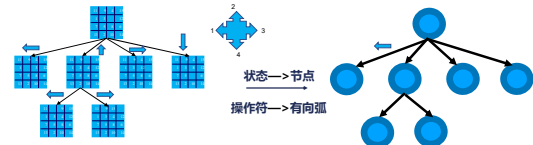
- **状态空间法的问题**：寻找从初始状态到目标状态的某一个操作符序列。
- **状态空间法的解**：从初始状态变换到目标状态的操作符序列。



状态图示法

当用有向图来表示状态空间法时，对应关系如下：

- ◆ 图中的一个节点对应于某一个状态。
- ◆ 图中的一个有向弧对应于某一个算符。
- ! 有向弧的旁边可以标以具体算符。



状态图示法

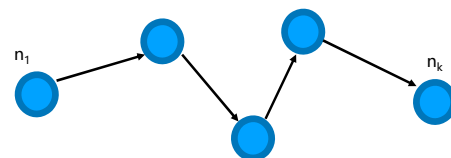
问题：寻找从初始状态到目标状态的某个操作符序列。



问题：寻找图中初始节点（对应初始状态）到目标节点（对应于目标状态）的一条路径。

状态图示法

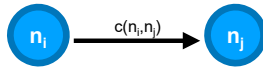
对于某一节点序列 $(n_1, n_2, \dots, n_j, \dots, n_k)$ ，当 $j = 2, 3, \dots, k$ 时，如果每一个节点 n_{j-1} 都有一个后继节点 n_j 存在，则将这一序列称为从节点 n_1 到 n_k 的**长度为 k 的路径**。



状态图示法

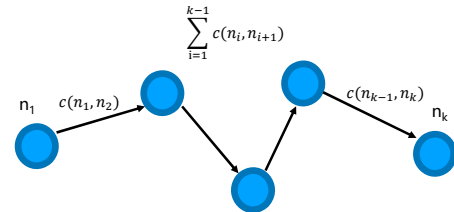
在某些情况下，每个操作符作用、成本是不一样的，需要引入代价的概念。

$c(n_i, n_j)$ 表示从节点 n_i 指向节点 n_j （相邻）的那一段弧的代价。



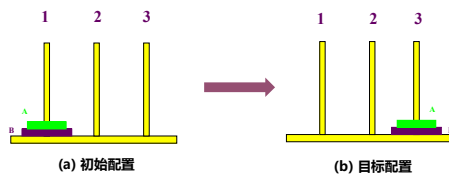
状态图示法

（不相邻的）两个节点间路径的代价等于连接该路径各个节点的所有弧线的代价之和：



状态空间法

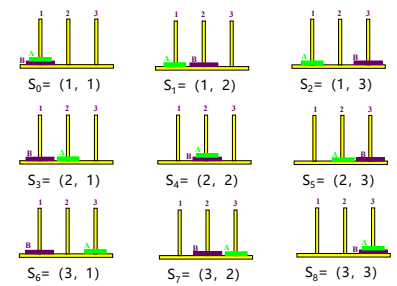
□ 梵塔问题（二圆盘）



状态空间法

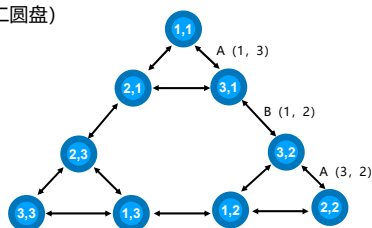
□ 梵塔问题（二圆盘）

- 状态： $S_k = (S_{K0}, S_{K1})$,
 S_{K0} 表示A所在的柱子编号， S_{K1} 表示B所在的柱子编号。
- 算符： A (1, 2)、A (1, 3)、B (2, 1)
 B (2, 3) ...



状态空间法

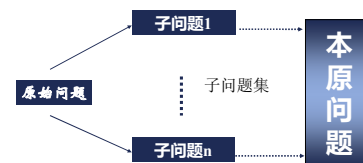
□ 梵塔问题（二圆盘）



二圆盘梵塔问题状态空间图

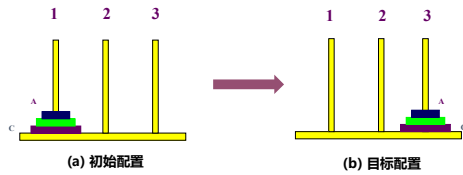
问题规约法

□ 讨论了问题归约的思路、实质、组成及与或图构建规则与示例分析。



梵塔问题

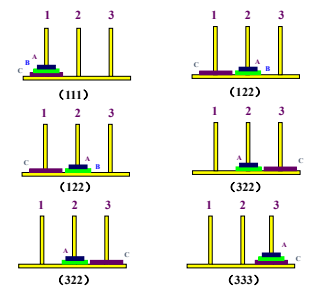
□ 梵塔问题（三圆盘）



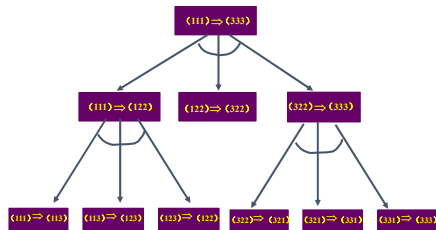
梵塔问题

□ 将原始问题归约为一个较简单问题集合

- (1) 移动圆盘A和B至柱子2的双圆盘问题；
- (2) 移动圆盘C至柱子3的单圆盘问题；
- (3) 大圆盘不能放在小圆盘上面。



梵塔问题归约图（与或图）



本章主题

1. 图搜索策略
2. 盲目搜索
3. 启发式搜索

本章主题

1. 图搜索策略
2. 盲目搜索
3. 启发式搜索

图搜索策略

□ 状态空间法与图的对应关系

状态空间中:		图中有:
状态	↔	节点
初始状态	↔	初始节点
目标状态	↔	目标节点
操作符	↔	有向弧

图搜索策略

- 在状态空间中，解是从初始状态到目标状态的操作符序列。
- 在图中，解是从初始节点到目标节点的一条路径。



19

2022/9/28

图搜索策略

- 图的搜索策略：它给出图搜索过程的一般步骤（基本思路、框架），经过细化后得到具体算法：
 - 盲目搜索技术（深度、宽度、代价优先算法）
 - 启发式搜索技术（有序算法、A*算法）



20

2022/9/28

图搜索策略

- 数据结构中图的遍历：从图某一个节点出发，访问遍图中其余节点，且每一个节点仅仅被访问一次。
- 当前图的搜索技术中，有两个特殊之处：
 - 搜索前，图并没有生成好，需要边生成边搜索。
 - 搜索从起始节点（初始状态）开始，到目标节点（目标状态）结束，不需要搜索所有可能的节点。

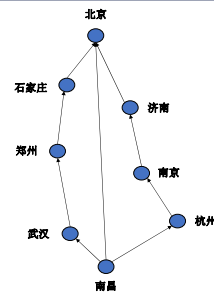


21

2022/9/28

图搜索过程

状态：(城市名)
 算子：南昌→武汉
 武汉↔郑州
 郑州↔石家庄
 石家庄↔北京
 ...
 初始状态：南昌
 目标状态：北京

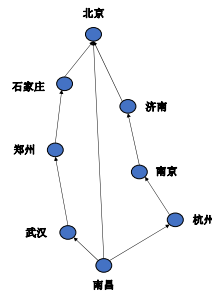


22

2022/9/28

图搜索过程

- 必须知道下一步还可以走哪些点
OPEN表(记录还没有扩展的点)
- 必须记住哪些点走过了
CLOSED表(记录已经扩展的点)
- 必须记住从目标返回的路径
每个表示状态的节点结构中必须有指向父节点的指针



23

2022/9/28

图搜索过程

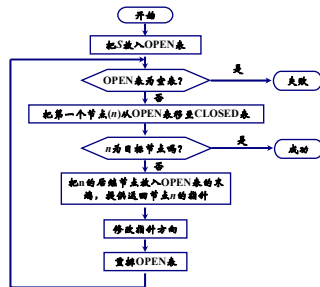
- 图搜索中的两个重要记号（符号）：
 - OPEN表：存放待扩展的节点。
 - CLOSED表：存放已扩展的节点。
- Tips!** 扩展节点：后继节点操作符（相当于可应用的规则）作用到节点（对应于某一状态描述）上，生成出其所有后继节点（新状态）并给出连接弧线的耗散值（相当于使用规则的代价），这个过程叫做扩展一个节点。



24

2022/9/28

图搜索过程



本章主题

1. 图搜索策略
2. 盲目搜索
3. 启发式搜索

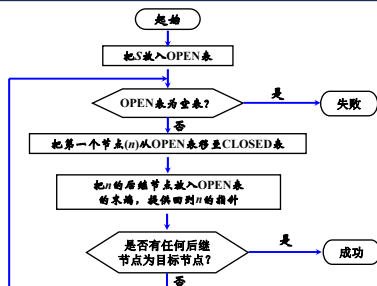
盲目搜索

- 盲目搜索是指无问题先验信息的搜索技术。
- 特点：
 - OPEN表中节点的排列是人为规定的
 - 一般只适合于求解比较简单的一些问题
- 种类：宽度优先、深度优先、等代价（代价优先）搜索等

宽度优先搜索

- 定义：以接近起始节点的程度逐层扩展节点的搜索方法（即：离起始节点近的节点先被拓展）。
- 扩展节点原则：先扩展出来的节点随后优先被扩展（生成其所有的后继节点）——先进先出（队列）
- 特点：一种高代价搜索，但若有解存在，则必能找到它。

宽度优先搜索

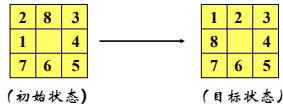


宽度优先搜索

- 注意几点：
 - ① 搜索过程产生的节点和指针构成一颗隐式定义的状态空间树的子树，称之为搜索树。
 - ② 宽度优先搜索方法能够保证在搜索树中找到一条通向目标节点的最短路径（所用操作符最少）。

宽度优先搜索

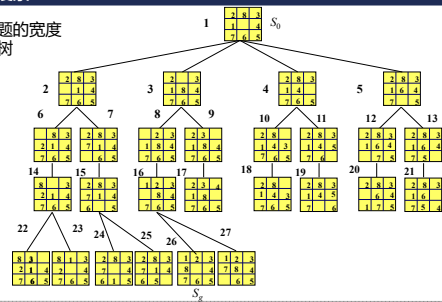
●八数码问题



规则：将牌移入空格的顺序为——从空格左边开始顺时针旋转。不许斜向移动，也不返回先辈节点。

宽度优先搜索

八数码问题的宽度 优先搜索树

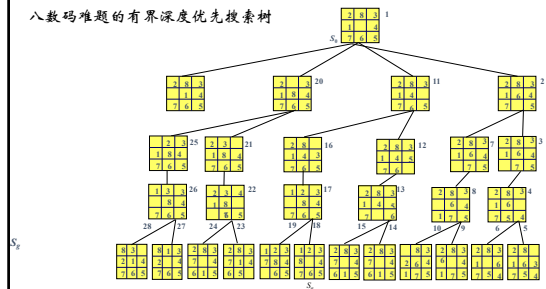


深度优先搜索

- 定义：首先扩展最新产生的(即最深的)节点——后进先出(栈)。
- 节点的深度：起始节点(即根节点)的深度为0，其他节点的深度规定为父节点深度加1，即 $d_{n+1}=d_n+1$ 。
- 基本思路：先扩展最深的节点，当出现没有后继节点时，换到旁边的次深节点。
- 算法：防止搜索过程沿着无益的路径扩展下去，往往给出一个节点扩展的最大深度——深度界限。
- 与宽度优先搜索最根本的不同在于：将扩展的后继节点放在OPEN表的前端。

深度优先搜索

八数码难题的有界深度优先搜索树



等代价搜索

- 宽度优先搜索技术找到了应用算符数目最少或者深度最浅的解。
 - 如果认为每一个操作符的代价相等，则宽度优先搜索技术就可以找到代价最小的路径或解。
- 例如：八数码问题以棋子的移动次数作为代价，每次状态的转换，代价相同，宽度优先可找到最优解。

等代价搜索

所有操作符代价相等



操作符的代价不相等

找到最短的路径



找到代价最小的路径

宽度优先算法



等代价(代价优先)搜索

等代价搜索

□ 推广的思路:

- 宽度优先搜索: 按照先后顺序取待扩展的节点。
- 等代价搜索: 将初始节点到所有端节点 (OPEN表中的节点) 的局部路径中代价最小的端节点作为下一个要扩展的节点。

□ 选取待扩展节点的原则:

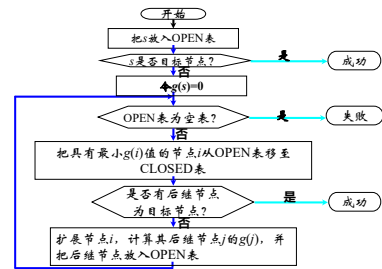
- 以 $g(i)$ (从起始节点 S 到某一节点 i 的路径的代价) 的递增顺序排列 OPEN 表。
- 以 $g(i)$ 最小的节点作为下一个要扩展的节点。



37

2022/9/28

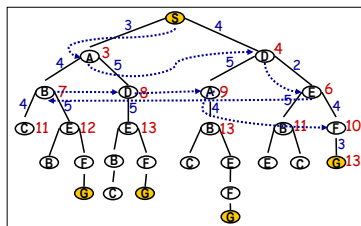
等代价搜索



38

2022/9/28

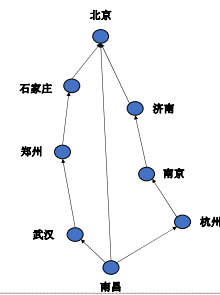
等代价搜索



39

2022/9/28

等代价搜索



40

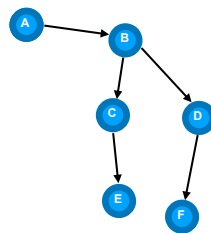
2022/9/28

宽度优先

扩展节点

Open表

Closed表



41

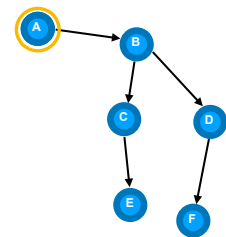
2022/9/28

宽度优先

扩展节点

Open表

Closed表



42

2022/9/28

宽度优先

扩展节点

A

Open表

B

Closed表

A

43 2022/9/28

宽度优先

扩展节点

B

Open表

C D

Closed表

A B

44 2022/9/28

宽度优先

扩展节点

C

Open表

D E

Closed表

A B C

45 2022/9/28

宽度优先

扩展节点

D

Open表

E F

Closed表

A B C D

46 2022/9/28

宽度优先

扩展节点

Open表

F

Closed表

A B C D E

47 2022/9/28

深度优先

扩展节点

Open表

Closed表

48 2022/9/28

深度优先

扩展节点

Open表

A

Closed表

49 2022/9/28

深度优先

扩展节点

A

Open表

B

Closed表

A

50 2022/9/28

深度优先

扩展节点

B

Open表

C D

Closed表

A B

51 2022/9/28

深度优先

扩展节点

D

Open表

C F

Closed表

A B D

52 2022/9/28

深度优先

扩展节点

F

Open表

C

Closed表

A B D F

53 2022/9/28

深度优先

扩展节点

C

Open表

E

Closed表

A B D F C

54 2022/9/28

深度优先

扩展节点

Open表

Closed表

A B D F C E

55 2022/9/28

等代价搜索

扩展节点

Open表

Closed表

56 2022/9/28

等代价搜索

扩展节点

Open表

A

Closed表

57 2022/9/28

等代价搜索

扩展节点

A

Open表

B

Closed表

A

58 2022/9/28

等代价搜索

扩展节点

B

Open表

C D

10 13

Closed表

A B

59 2022/9/28

等代价搜索

扩展节点

C

Open表

D E

13 18

Closed表

A B C

60 2022/9/28

等代价搜索

扩展节点

D

Open表

F E
17 18

Closed表

A B C D

61 2022/9/28

等代价搜索

扩展节点

F

Open表

E
18

Closed表

A B C D F

62 2022/9/28

等代价搜索

扩展节点

Open表

Closed表

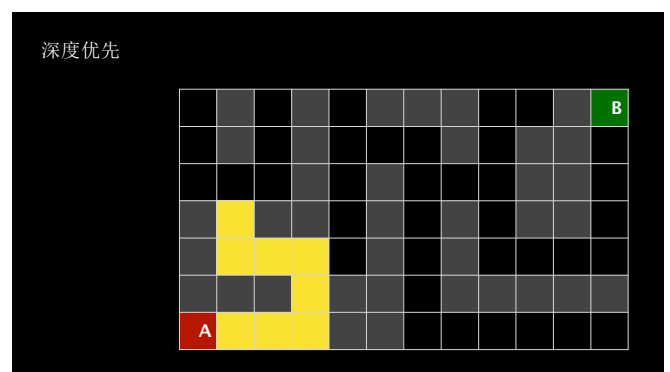
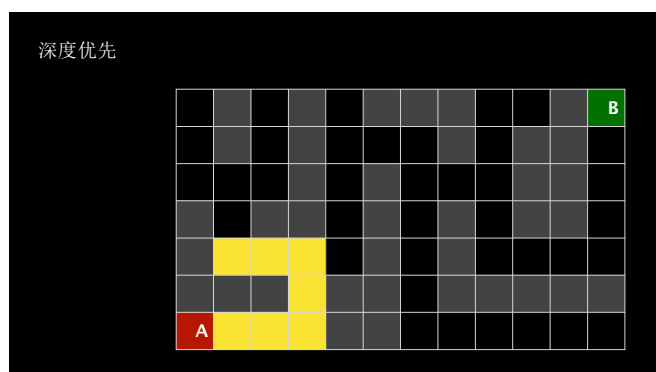
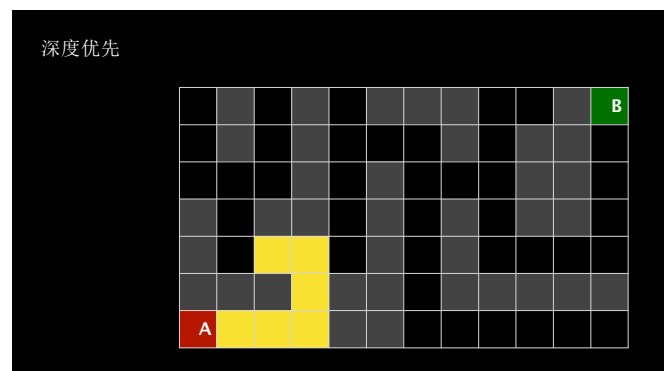
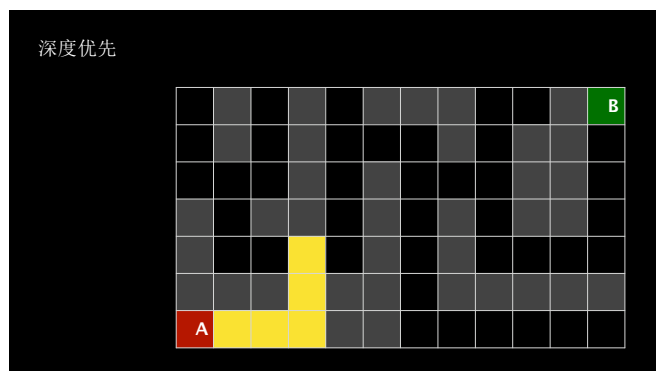
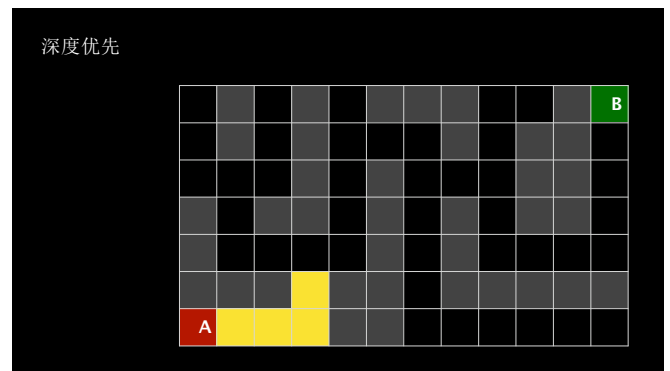
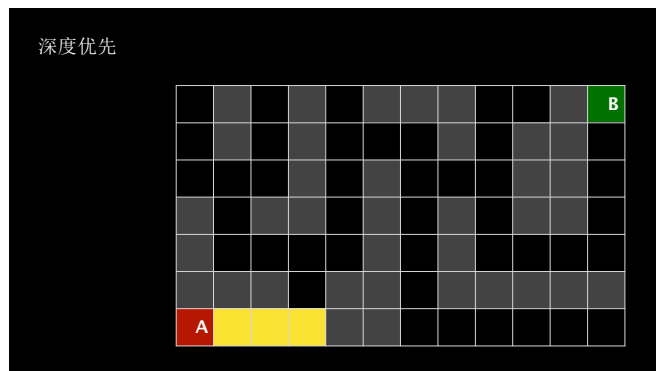
A B C D F E

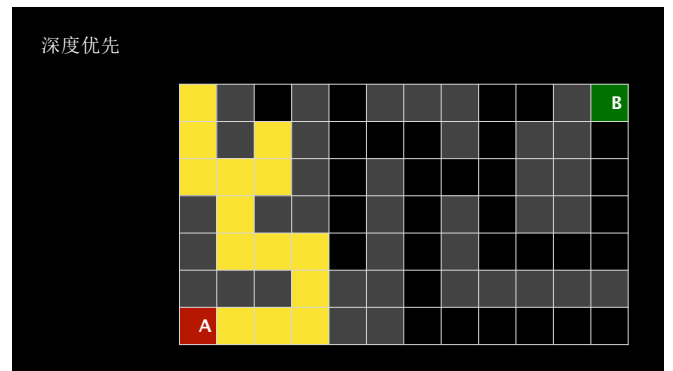
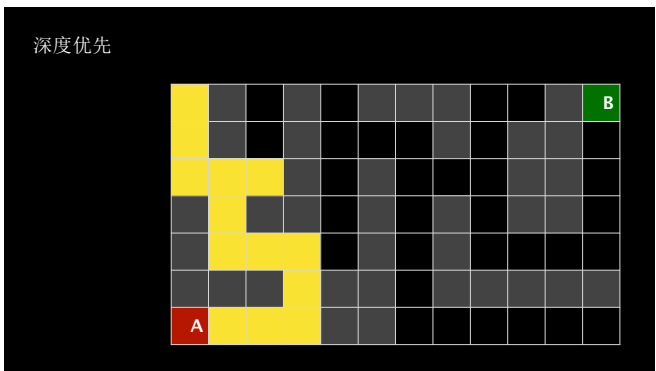
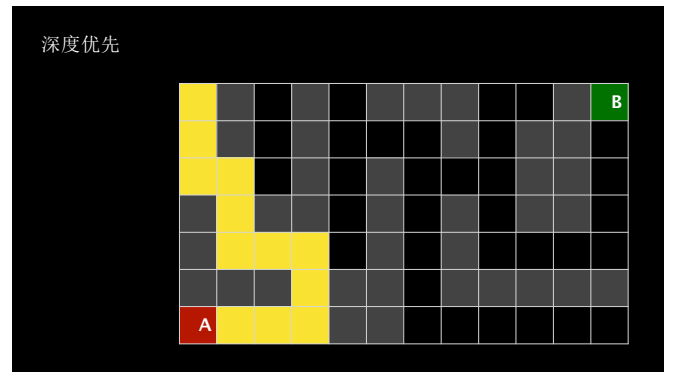
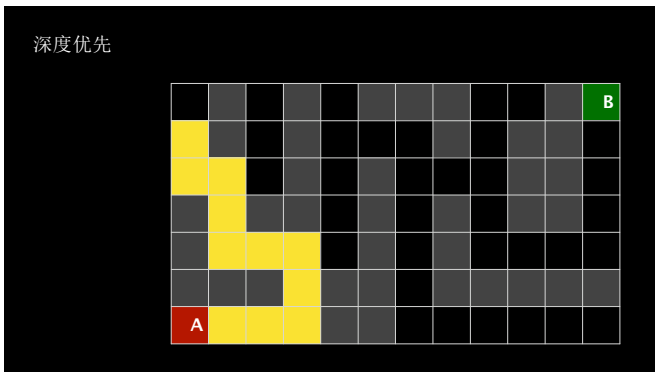
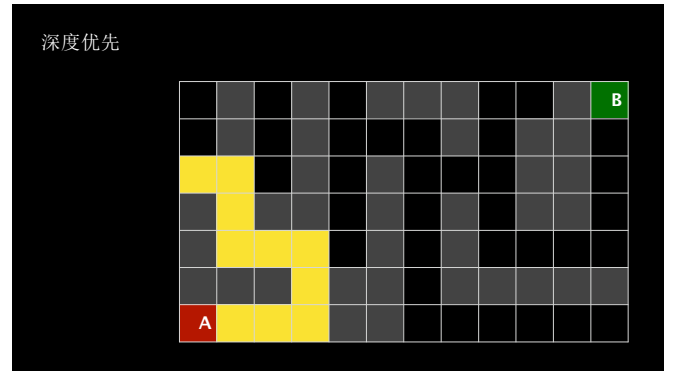
63 2022/9/28

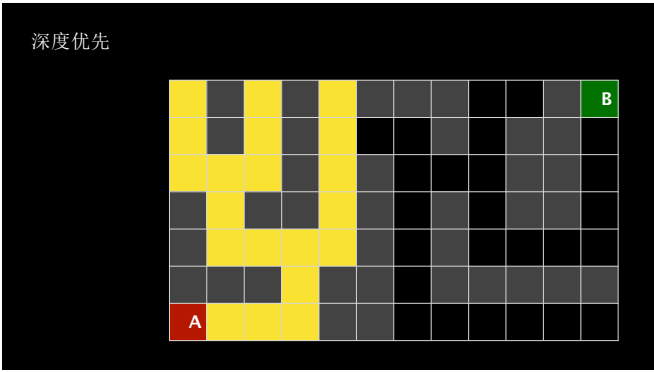
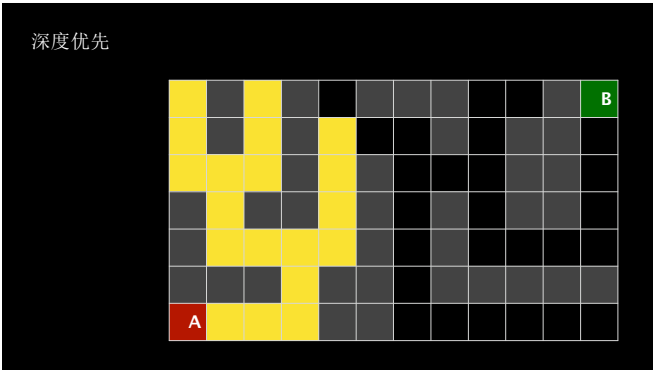
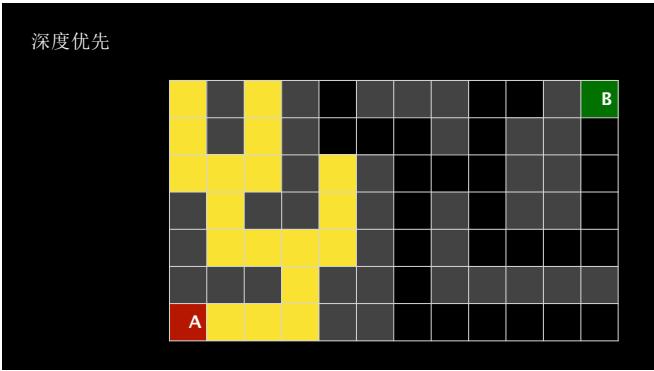
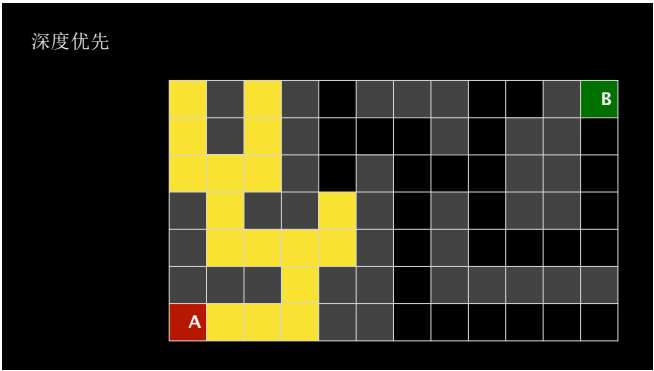
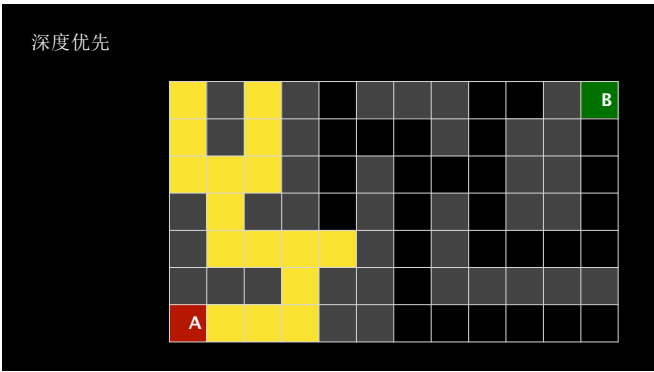
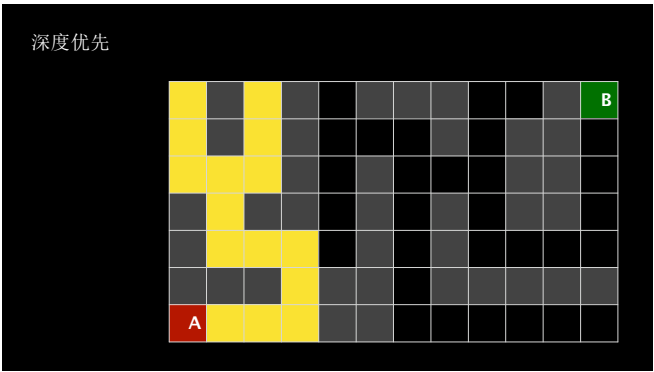
深度优先

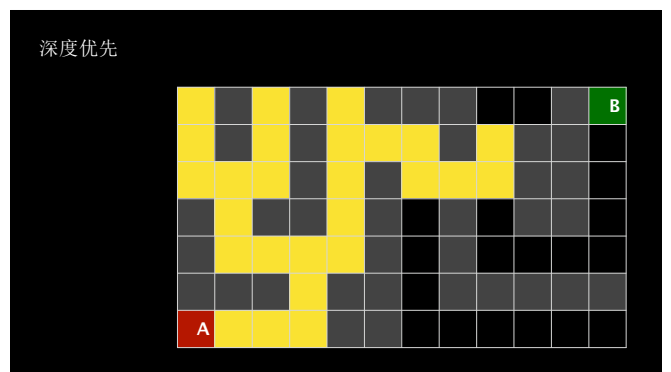
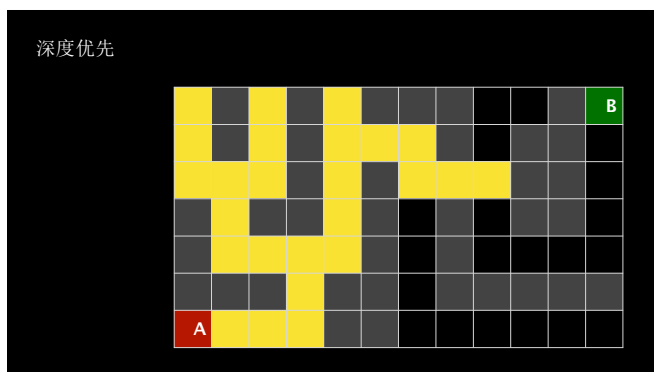
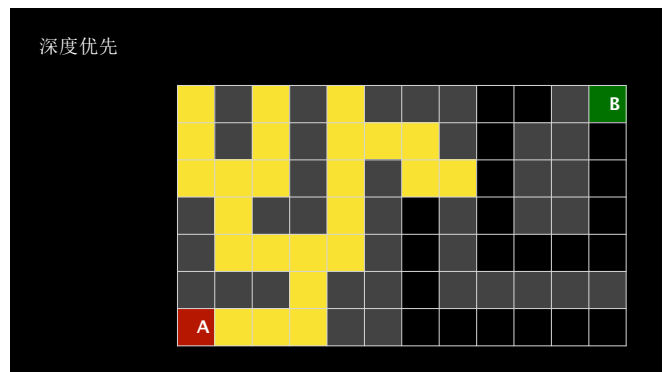
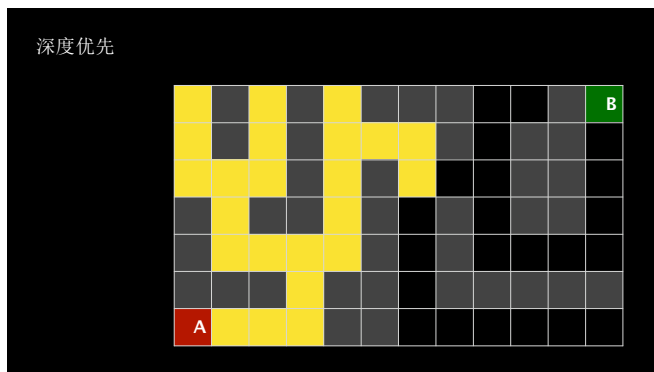
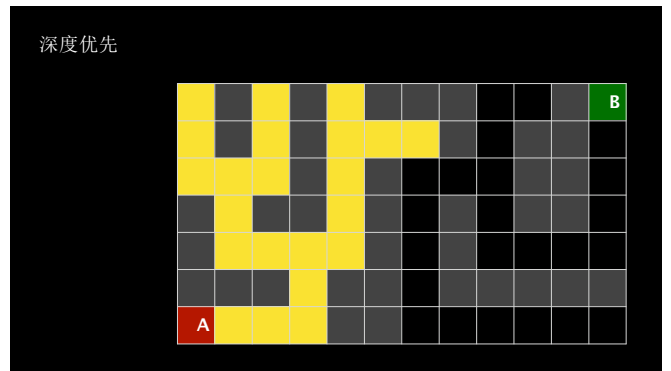
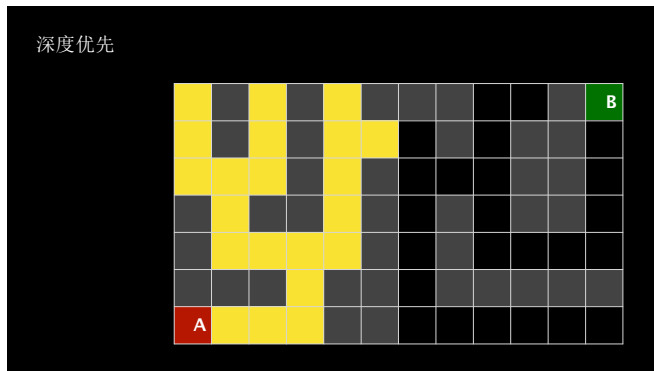
深度优先

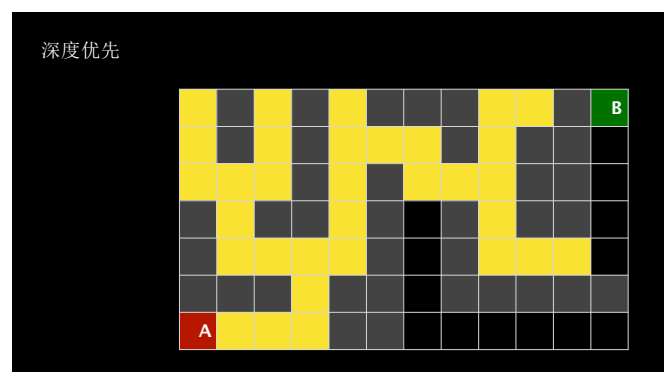
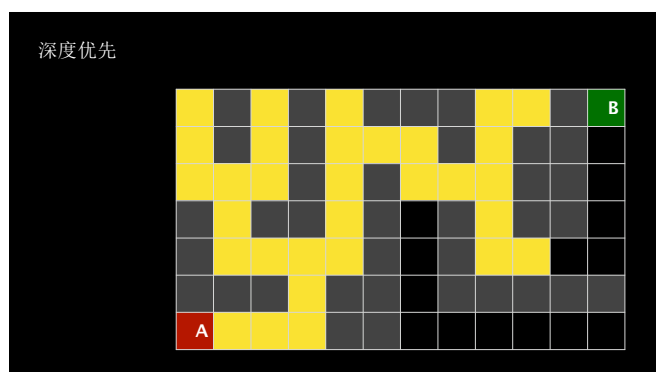
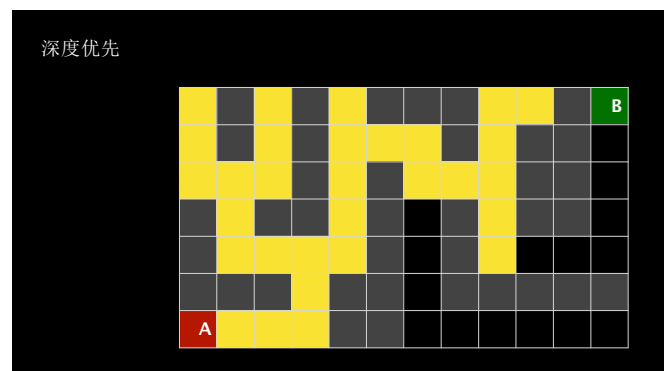
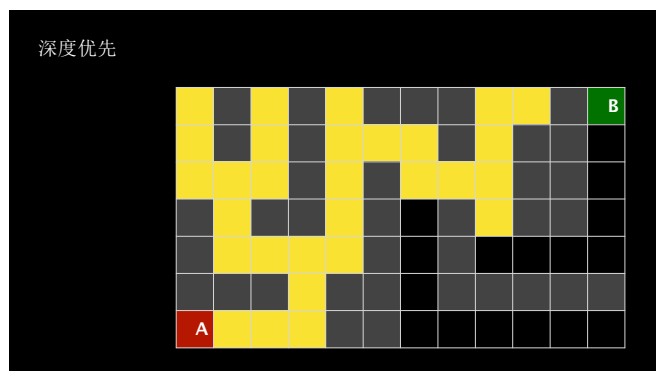
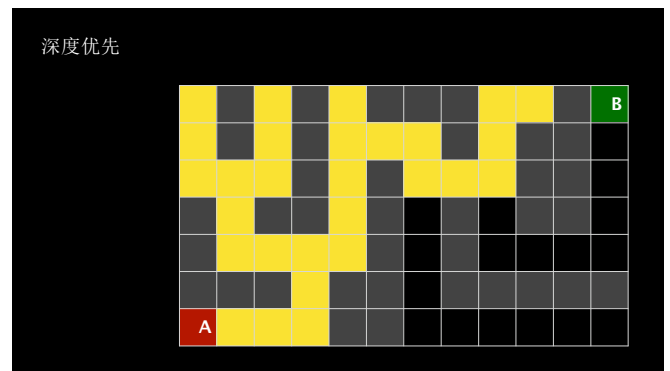
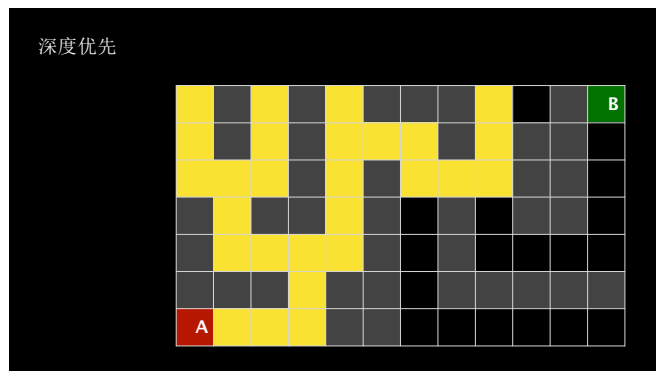
深度优先



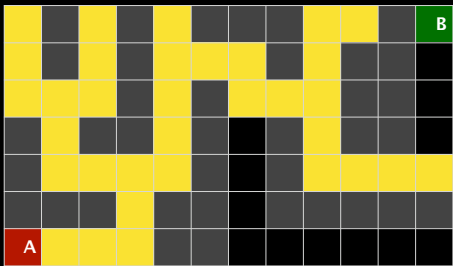




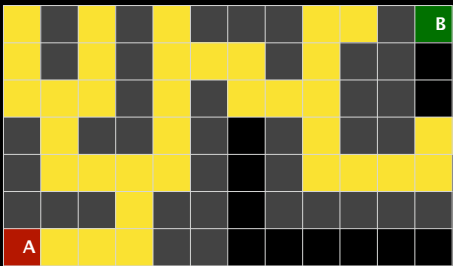




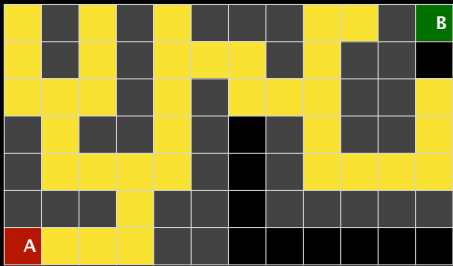
深度优先



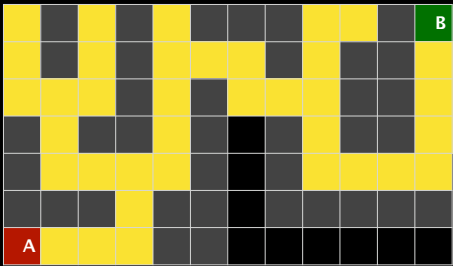
深度优先



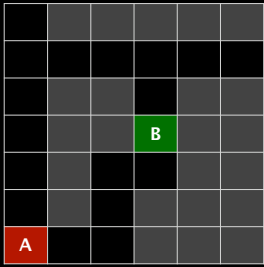
深度优先



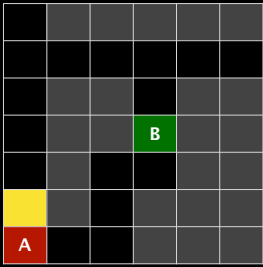
深度优先



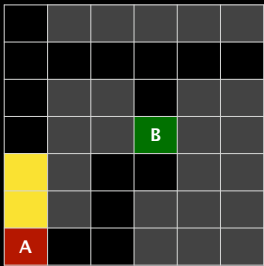
深度优先



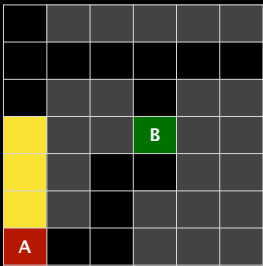
深度优先



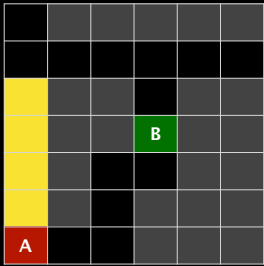
深度优先



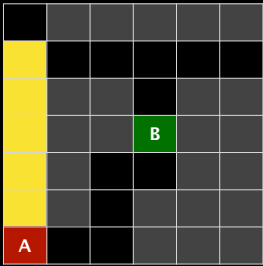
深度优先



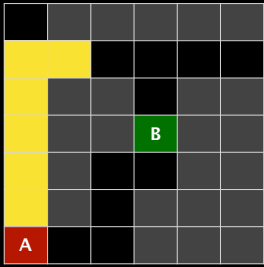
深度优先



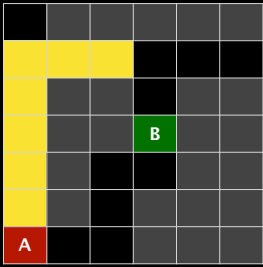
深度优先



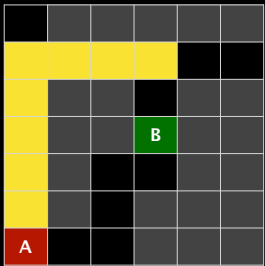
深度优先



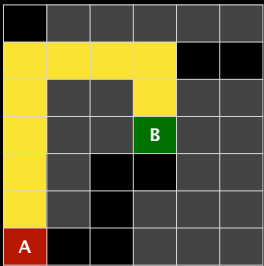
深度优先



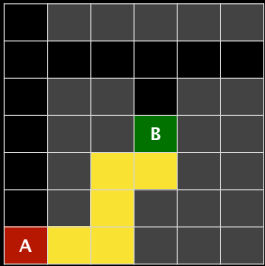
深度优先



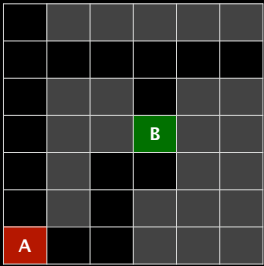
深度优先



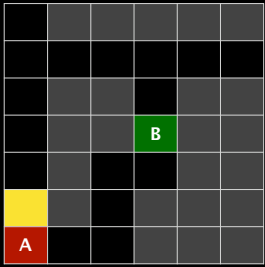
深度优先



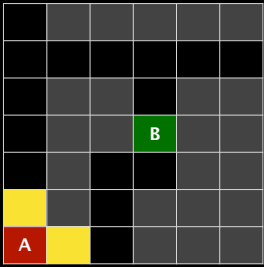
宽度优先



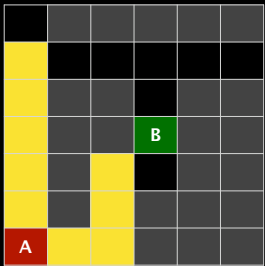
宽度优先



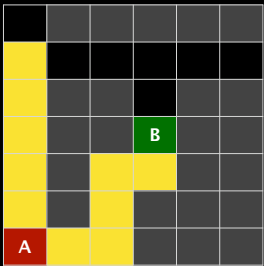
宽度优先



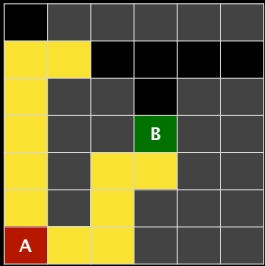
宽度优先



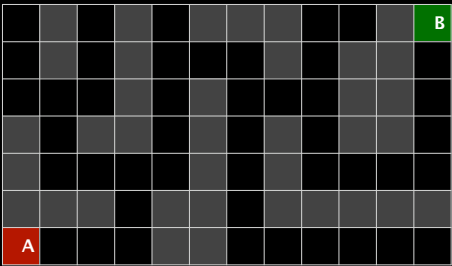
宽度优先



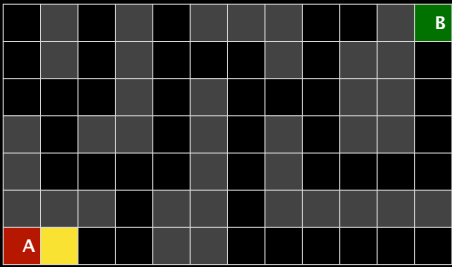
宽度优先



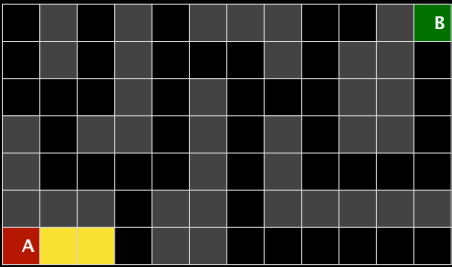
宽度优先



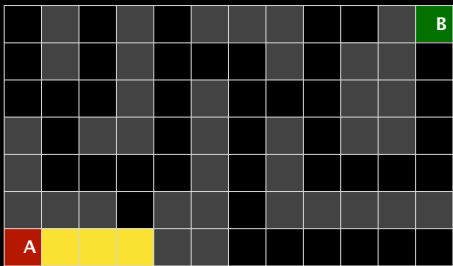
宽度优先



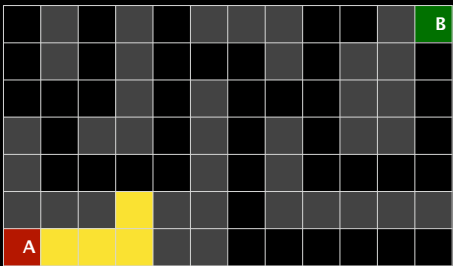
宽度优先



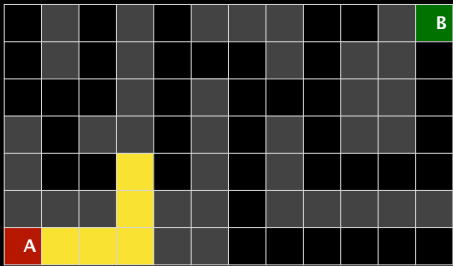
宽度优先



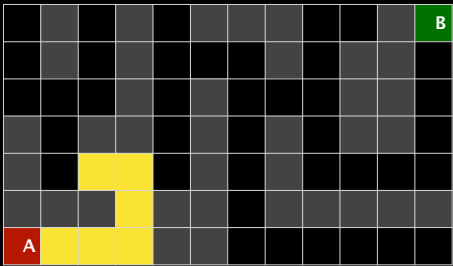
宽度优先



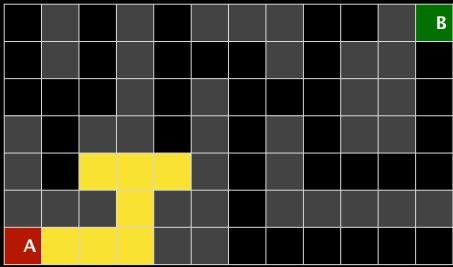
宽度优先



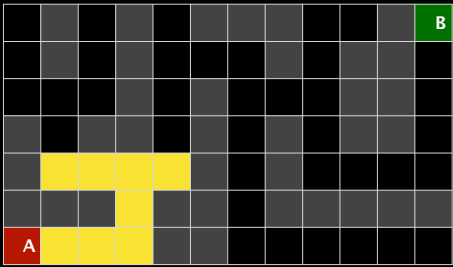
宽度优先

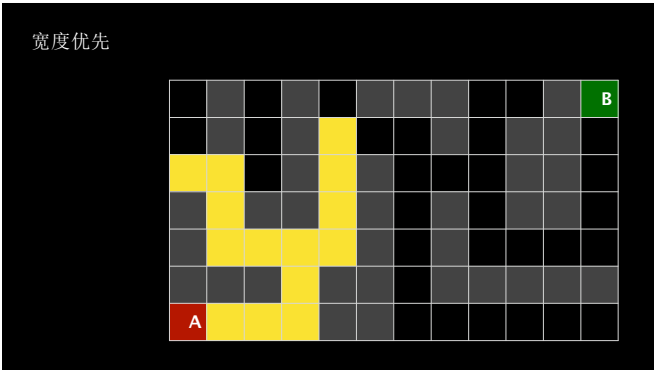
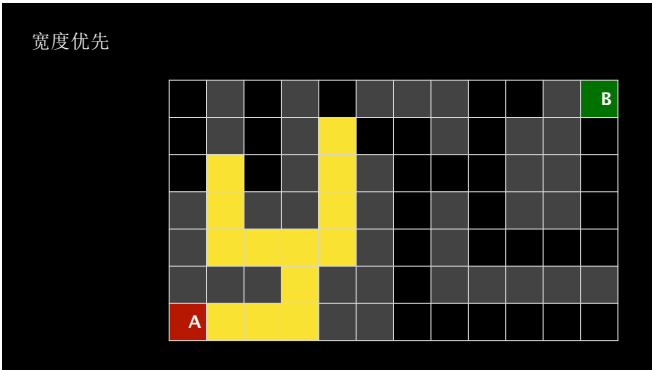
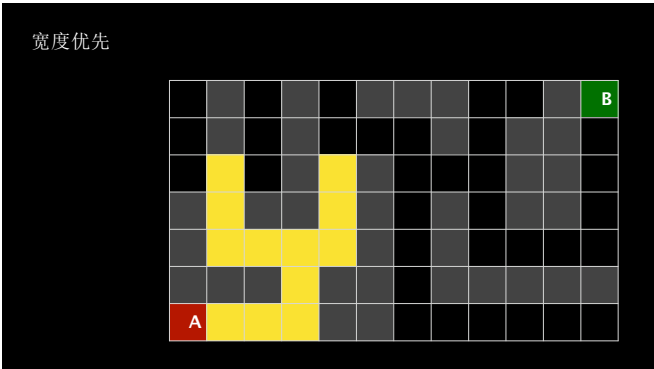
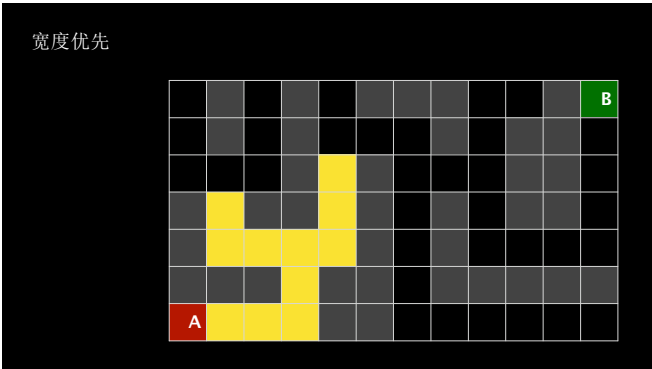
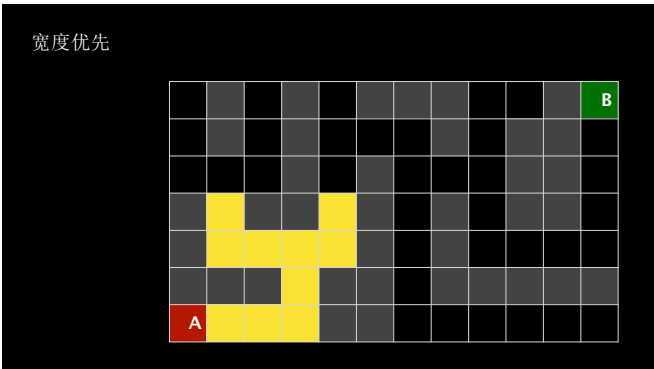
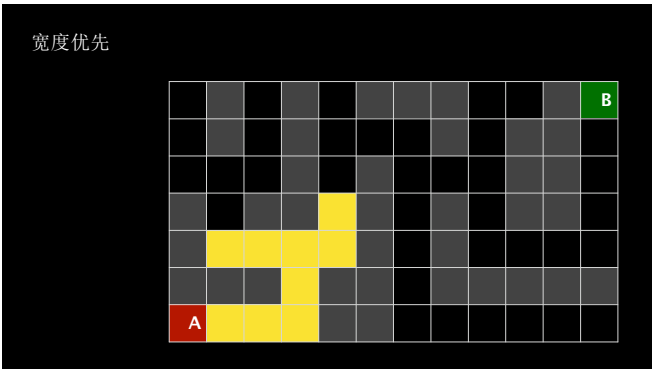


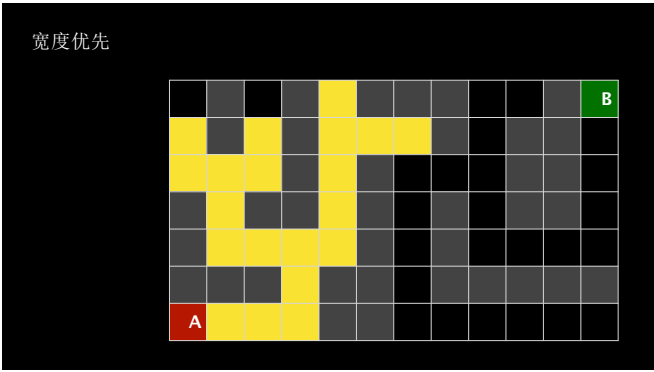
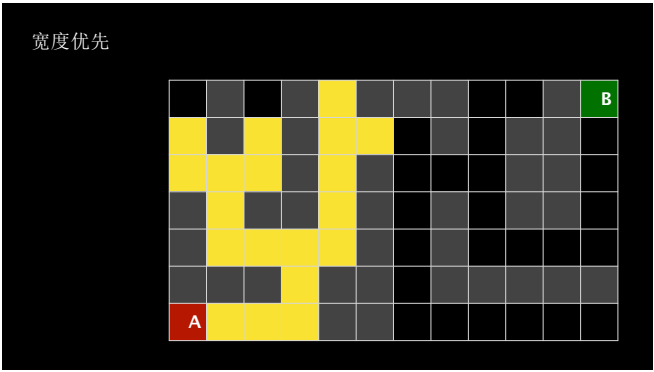
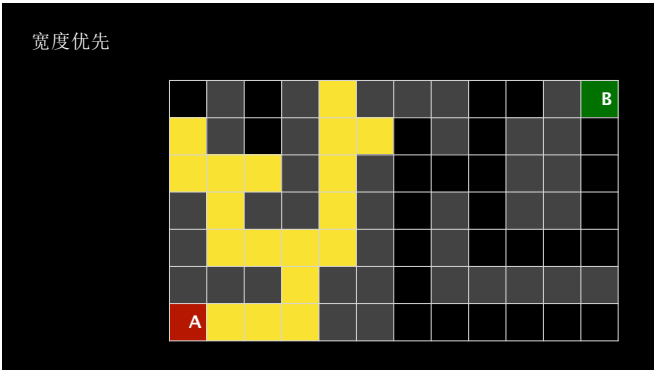
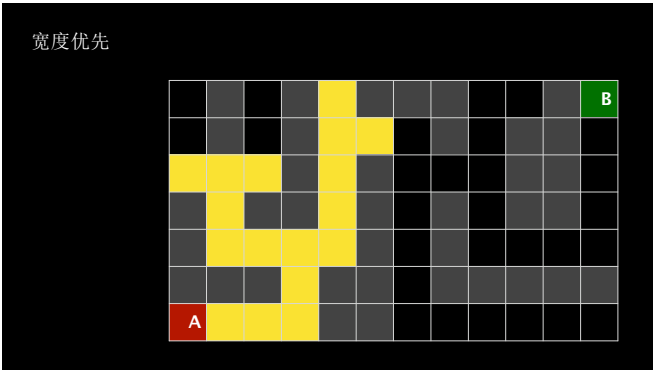
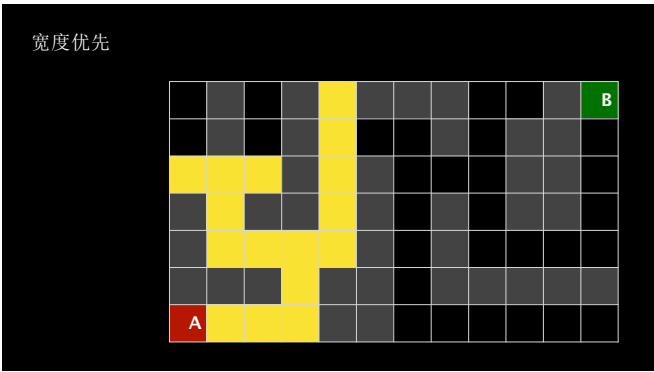
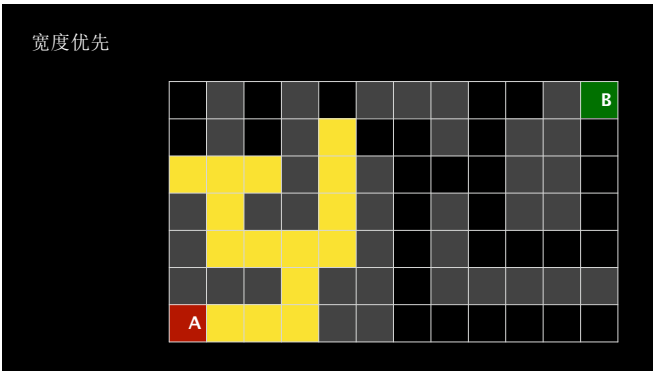
宽度优先

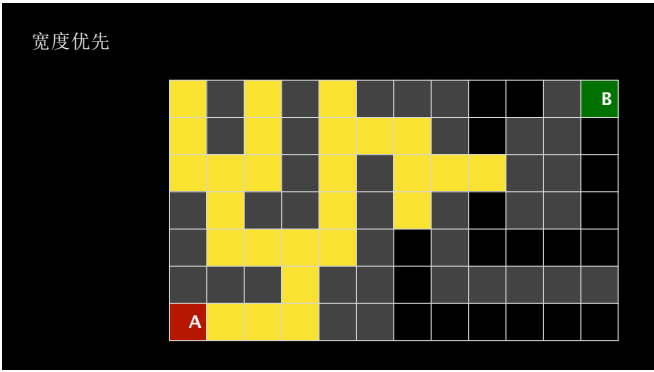
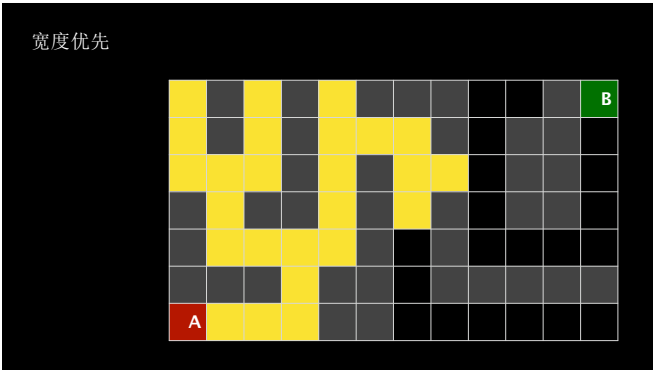
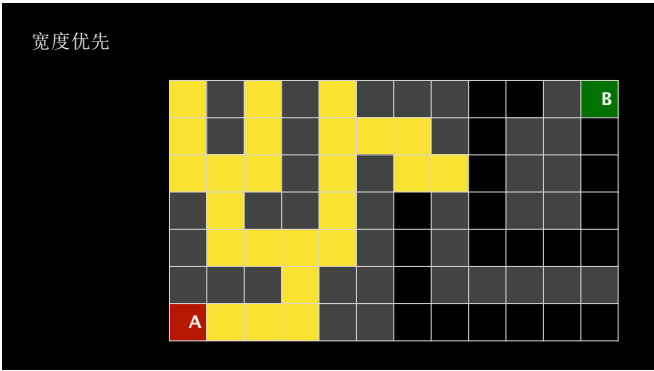
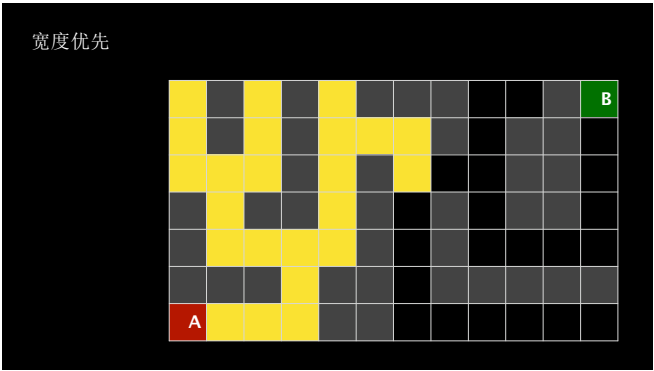
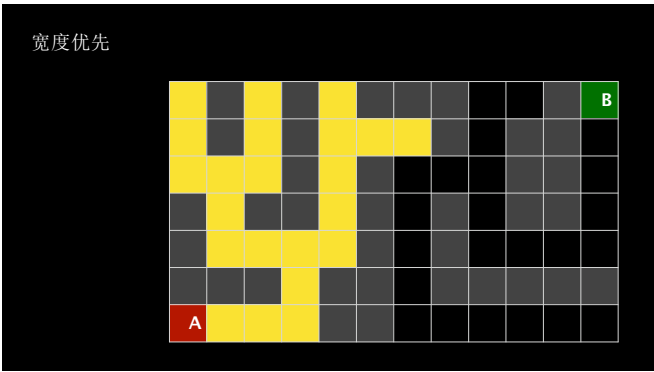
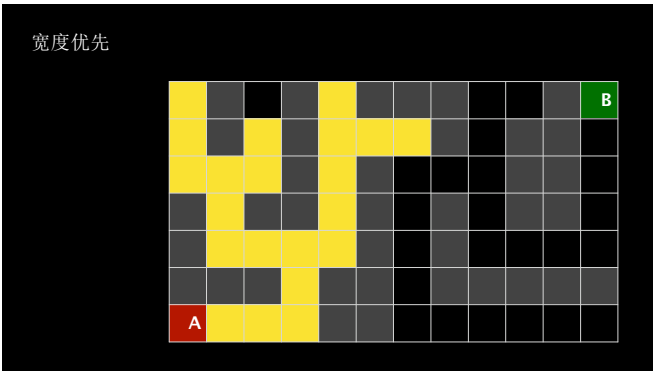


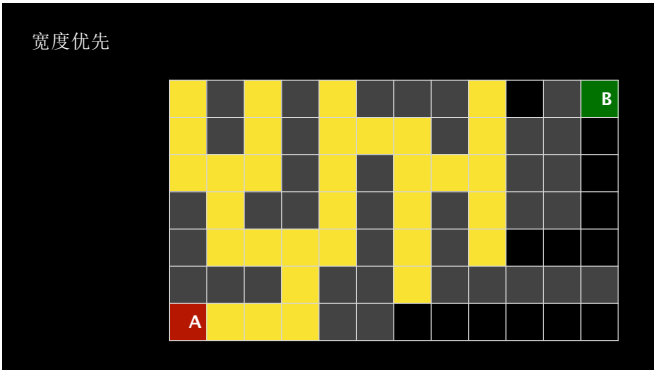
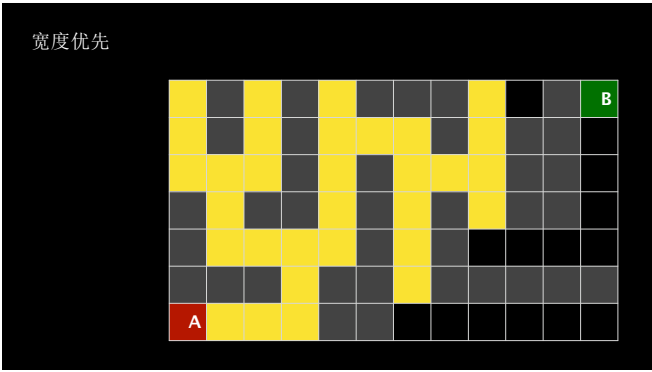
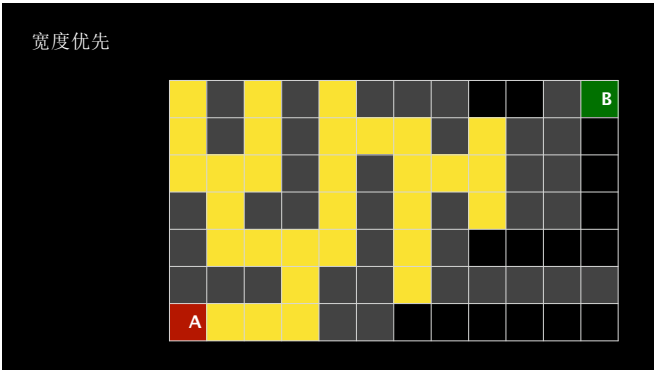
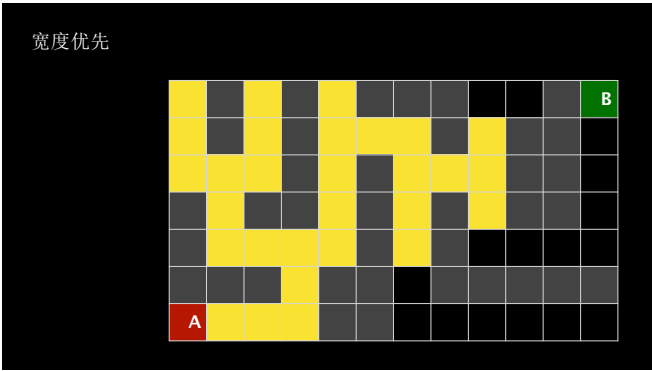
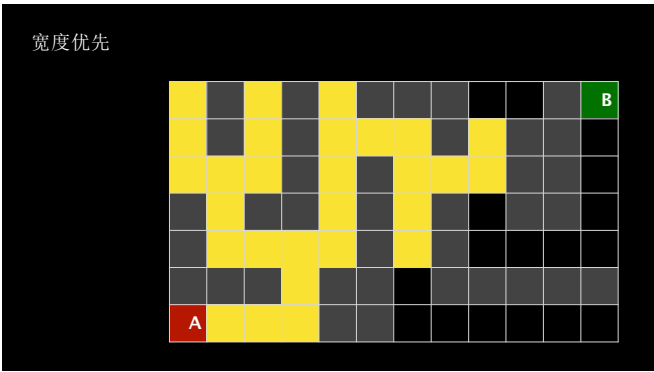
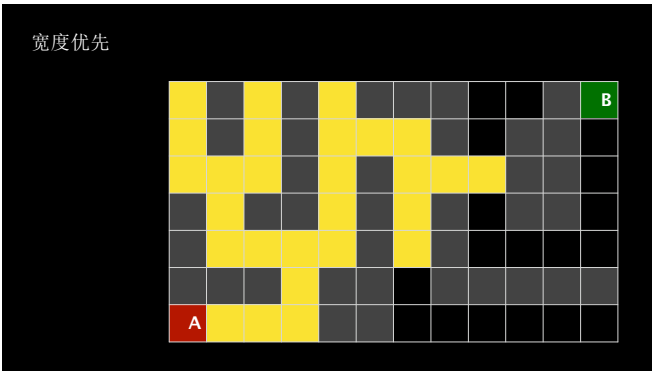
宽度优先

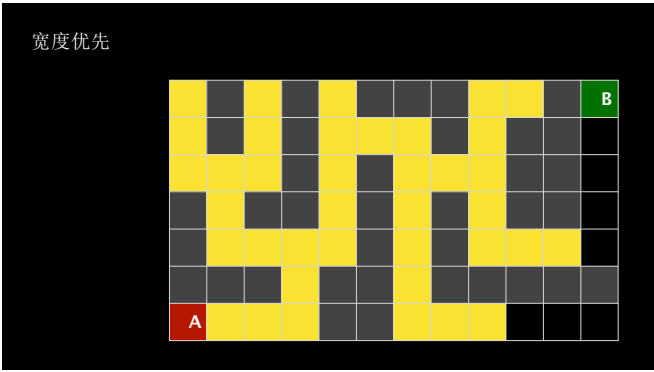
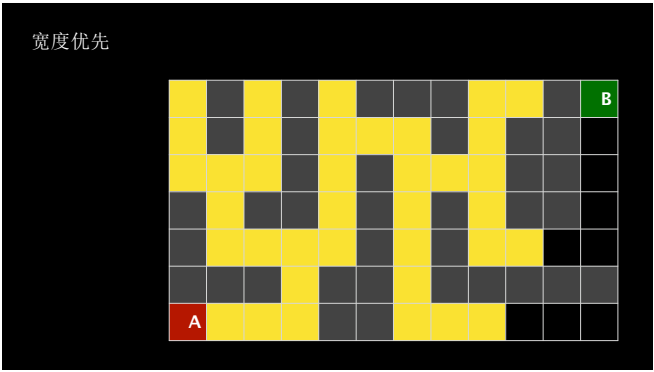
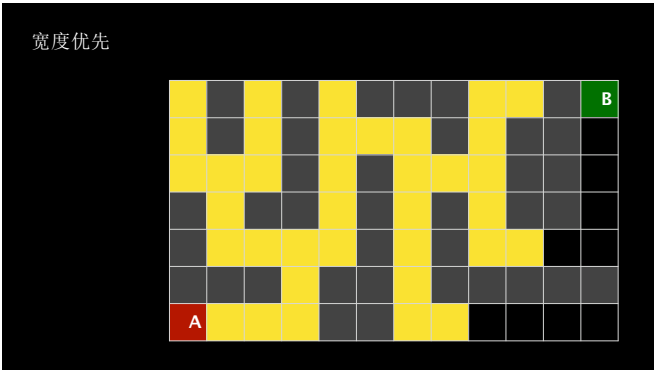
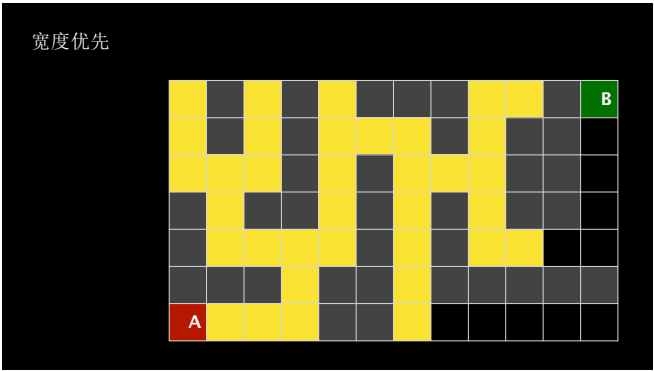
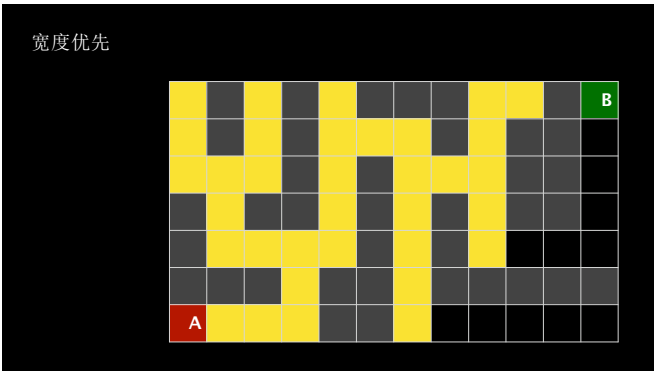
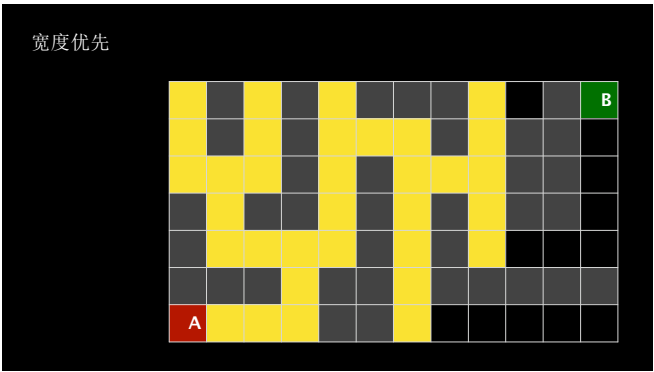




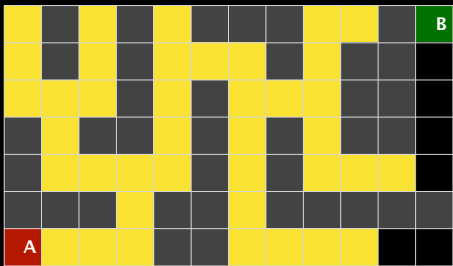




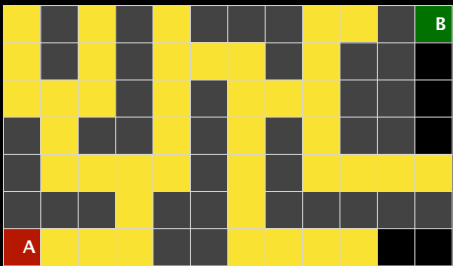




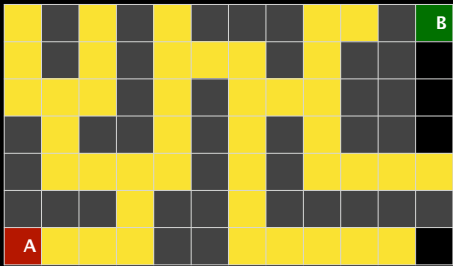
宽度优先



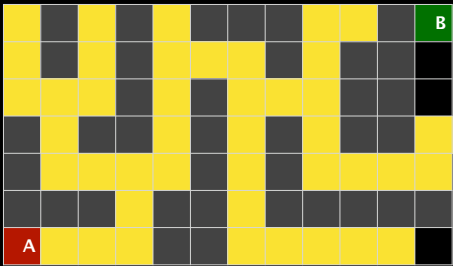
宽度优先



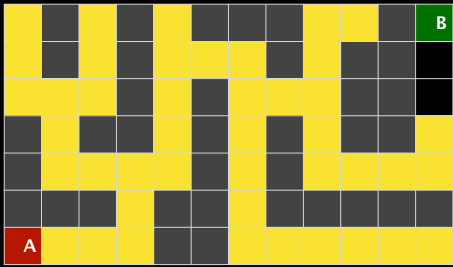
宽度优先



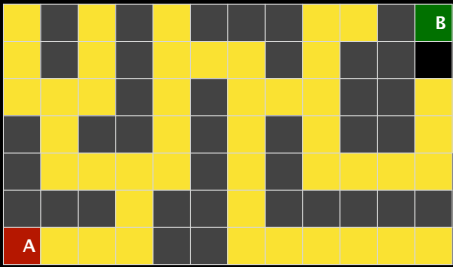
宽度优先



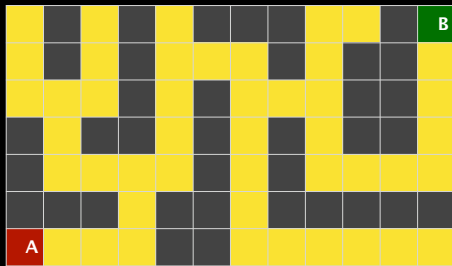
宽度优先



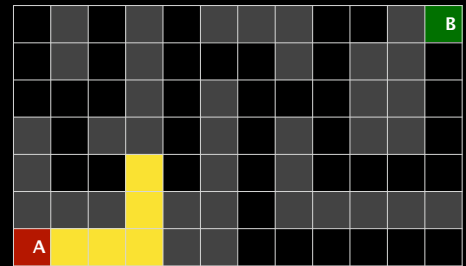
宽度优先



宽度优先



宽度优先



是什么影响了搜索效率

●八数码问题

2	8	3
1		4
7	6	5

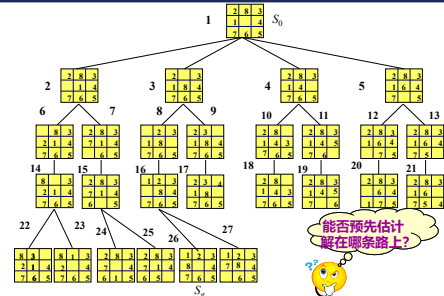
(初始状态)

1	2	3
8		4
7	6	5

(目标状态)

操作: 空格上移, 空格下移, 空格左移, 空格右移

是什么影响了搜索效率



本章主题

1. 图搜索策略
2. 盲目搜索
3. 启发式搜索

启发式搜索

- 宽度和深度优先的缺点: 扩展节点的顺序是人为规定的, 要扩展节点的数目可能非常大, 占用大量的计算时间和内存空间, 使得搜索效率低。
- 等代价搜索技术的缺点: 选取已经搜索到的代价最小的节点来扩展, 但是没有考虑目标状态, 不知道离目标状态还有多远。
- 提高搜索效率的思路: 利用更多的与问题有关的信息来选取待扩展的节点。

启发式搜索策略

- **启发信息（背景知识）**：用来加速搜索过程的有关问题领域的特征信息——与具体问题特性有关的一些信息（在具体的使用过程中，还与使用者、算法有关）。
- **启发式搜索方法**：利用启发信息的搜索方法。



175

2022/9/28

启发式搜索策略

□ 搜索技术中启发信息的用途：

- ① 用于决定要扩展的下一个节点，避免宽度优先或深度优先搜索中盲目（人为规定）地选择扩展节点。
- ② 在扩展一个节点的过程中，用于决定要生成哪一个或哪几个后继节点，避免盲目地同时生成所有可能地后继节点。
- ③ 用于决定某些应该从搜索树中抛弃或修剪的节点。



176

2022/9/28

启发式搜索

- **有序搜索**：在搜索过程中总是选择“最有希望”的节点作为下一个被扩展节点的搜索技术。
- **估价函数**：用来估算节点“希望”程度的函数。



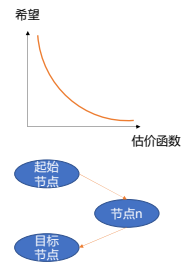
177

2022/9/28

启发式搜索

□ 估价函数的基本特性：

- ① 一般情况下，估价函数值越大，希望程度就越低。
- ② 根据搜索过程、问题的启发信息来定义的，对搜索效率会产生较大的影响。
- ③ 用 $f(n)$ 来表示节点 n 的估价函数值，并且期望，它是从初始节点、通过节点 n 、到达目标节点的最小代价的一个估计值。



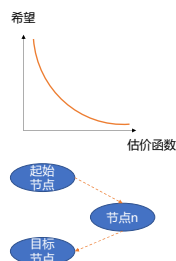
178

2022/9/28

启发式搜索

□ 估价函数的基本特性：

- ① 一般情况下，估价函数值越大，希望程度就越低。
- ② 根据搜索过程、问题的启发信息来定义的，对搜索效率会产生较大的影响。
- ③ 用 $f(n)$ 来表示节点 n 的估价函数值，并且期望，它是从初始节点、通过节点 n 、到达目标节点的最小代价的一个估计值。



179

2022/9/28

启发式搜索

□ 估价函数的计算：计算一个节点的“估价函数”，可以分成两个部分。

- ① 已经付出的代价（起始节点到当前节点）。
- ② 将要付出的代价（当前节点到目标节点）



180

2022/9/28

启发式搜索

节点 n 的估价函数 $f(n)$ 定位为从初始节点、经过 n 、到达目标节点的路径的最小代价的估计值，即

$$f(n) = g(n) + h(n)$$

$g(n)$ 是从初始节点到达当前节点 n 的实际代价；

$h(n)$ 是从节点 n 到目标节点的最佳路径的估计代价



启发式搜索

$$f(n) = g(n) + h(n)$$

$h(n)$ 体现出搜索过程中采用的启发式信息（背景知识），称之为启发函数。 $g(n)$ 所占比重越大，越趋向于宽度优先或等代价搜索；反之， $h(n)$ 的比重越大，表示启发性能就越强。



启发式搜索

引入**估价函数**(evaluation function)来估计节点位于解路径上的“希望”，函数值越小“希望”越大。

$$f(x) = g(x) + h(x)$$

从起始状态到当前状态的估计代价 (启发函数)

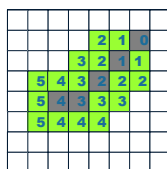
搜索过程中按照估价函数的大小对OPEN表排序，每次选择**估价函数值最小**的节点作为下一步考察的节点。

与前面将搜索策略统一到“图一般搜索策略”的框架下类似，将一般策略稍作修改，可建立有信息搜索的一般搜索策略——“最佳优先搜索(Best First Search)”，也称有序搜索。

启发式搜索

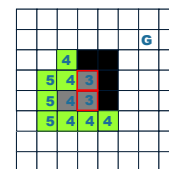
	10	9	8	7	6	5	4	3	2	1	B
	11										1
	12		10	9	8	7	6	5	4		2
	13		11						5		3
	14	13	12		10	9	8	7	6		4
			13		11						5
A	16	15	14		12	11	10	9	8	7	6

启发式搜索



优先选择距离目标最近的点

启发式搜索



启发式搜索

□ 在GRAPHSEARCH过程中，如果第8步的重排OPEN表是依据 $f(x)=g(x)+h(x)$ 进行的，则称该过程为A*算法。

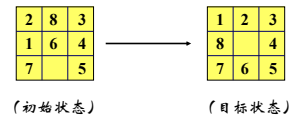
□ 估价函数满足一定限制条件的算法称为A*算法。

$$f(x) = g(x) + h(x)$$

大于0 不大于x到目标的实际代价

启发式搜索

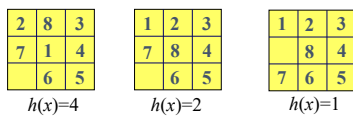
□ 八数码问题举例



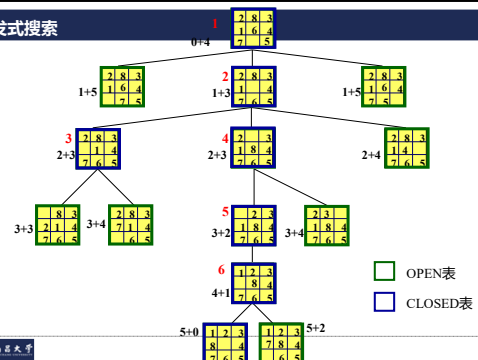
启发式搜索

□ $g(x)$: 从初始状态到x需要进行的移动操作的次数。

□ $h(x)$: 错放的棋子数。



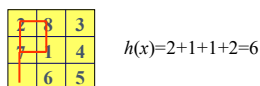
启发式搜索



启发式搜索

□ $g(x)$: 从初始状态到x需要进行的移动操作的次数。

□ $h(x)$: 所有棋子与目标位置的曼哈顿距离之和。



启发式搜索

